

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 1 – HANDS-ON API ASSIGNMENT (HUGGING FACE / OPENAI / GEMINI)

Course Code:	CSA65	Course Title:	Generative AI and Large Scale Models
CO Assessed:	CO3 – Precision (BL2, BL3)	Assessment:	Assessment Tool 1 – Hands-on API Assignment (Hugging Face / OpenAI / Gemini)
Weightage:	20% of CIA	Total Marks:	2 * 50 = 100 (1 Question1)
CO3 Statement:	<i>Develop intelligent applications using Retrieval-Augmented Generation (RAG), vector databases, and introductory AI agent concepts.(BL3, BL4)</i>		
Student Name:		Reg. No.:	
Section / Batch:		Date:	

Instructions to Students

- Answer 2 questions. Each question carries 50 marks.Total: 100 Marks.
- Use Python to implement the assigned problem.
- Use at least one API/platform: Hugging Face, OpenAI, or Google Gemini.
- Accept suitable user input and clearly display the generated output.
- Handle API errors, invalid input, and other suitable edge cases.
- Use readable, structured, and modular code wherever appropriate.
- Add comments and brief documentation explaining the implementation.
- Demonstrate the program with suitable test cases.
- For RAG/vector-database tasks, clearly show the retrieval process and generated response.

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

SECTION A – HANDS-ON API ASSIGNMENT (HUGGING FACE / OPENAI / GEMINI)

5. Blog Content Generator: Create an application using an LLM API that generates a short blog article based on a topic entered by the user.

Q1. Blog Content Generator Using Gemini API

Aim

To develop a Python-based Blog Content Generator that uses the Google Gemini Large Language Model (LLM) API to generate a short and informative blog article based on a topic entered by the user.

Objective

The main objectives of this application are:

- To accept a blog topic from the user.
- To integrate the Google Gemini API with Python.
- To generate meaningful blog content using an LLM.
- To structure the generated content with a title, introduction, sections, and conclusion.
- To handle invalid or empty user input.
- To handle API-related errors.
- To demonstrate the application using suitable test cases.

Technologies Used

Technology	Purpose
Python	Application development
Google Gemini API	Blog content generation
Google Colab	Development and execution environment
google-genai	Python SDK for Gemini API

Problem Description

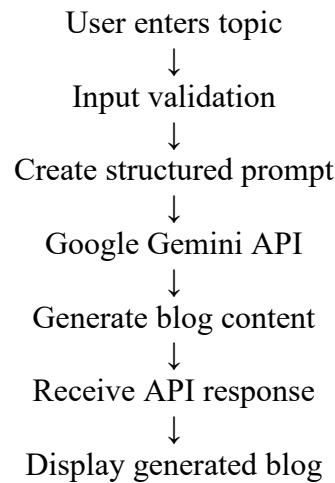
Writing a good blog article manually requires time and effort. The proposed system automates the initial content-generation process by using a Large Language Model. The user provides a topic, and the application sends a structured prompt to the Gemini API. The generated response is then displayed as a short blog article.

Working Principle

The system works through the following steps:

1. The user enters a topic for the blog.
2. The program validates the entered topic.
3. A structured prompt is created using the topic.
4. The prompt is sent to the Google Gemini API.
5. Gemini processes the prompt and generates the blog content.
6. The generated response is received by the Python program.
7. The blog is displayed to the user.
8. If the input is empty or an API error occurs, an appropriate error message is displayed.

System Flow:



Implementation

The application was implemented in Python using the google-genai library. A Gemini client was created using the API key, and the `generate_content()` method was used to send the prompt and receive the generated response.

The application contains a separate `generate_blog()` function to keep the code modular. The function accepts the topic as input, creates a prompt, sends it to Gemini, and returns the generated blog.

The program also uses exception handling to prevent the application from terminating unexpectedly when an API error occurs.

```
import os

# Create Gemini client
client = genai.Client(api_key=os.environ["GEMINI_API_KEY"])

def generate_blog(topic):
    """
    Generates a short blog article based on the topic
    entered by the user.
    """
    prompt = f"""
    Create a short blog article about: {topic}

    Use exactly this structure:

    TITLE:
    Write a catchy title.

    INTRODUCTION:
    Write a short introduction.

    SECTION 1:
    Give a suitable heading and explain the first important point.

    SECTION 2:
    Give a suitable heading and explain the second important point.

    SECTION 3:
    Give a suitable heading and explain the third important point.

    CONCLUSION:
    Write a short conclusion.

    Requirements:
    - Write approximately 400-500 words.
    - Use simple and clean English.
    - Make the content informative and engaging.
    - Keep each section well organized.
    - Do not use unnecessary emojis.
    - Do not mention that the article was generated by AI.
    """

    try:
        response = client.models.generate_content(
            model="gemini-2.5-flash",
            contents=prompt
        )
        return response.text
    except Exception as e:
        return f"API Error: {e}"

# Get topic from user
topic = input("Enter a topic for your blog: ").strip()
```

Figure 1: Python implementation of the Blog Content Generator

Prompt Used

The application uses a structured prompt similar to the following:

Create a short blog article about: <topic>

Use the following structure:

TITLE
INTRODUCTION
SECTION 1
SECTION 2
SECTION 3
CONCLUSION

Requirements:

- Write approximately 400–500 words.
- Use simple and clear English.
- Make the content informative and engaging.
- Keep each section well organized.

This prompt helps the LLM generate content in a consistent format.

Input

The application accepts a topic from the user.

Example Input:

Artificial Intelligence in Education

Another test input:

Cybersecurity for College Students

```
print(topic)
print("-" * 60)

... Enter a topic for your blog: Artificial Intelligence in Education

Generating blog...
-----

GENERATED BLOG
=====
TITLE: AI in the Classroom: Unlocking a Smarter Future for Learning

INTRODUCTION:
Artificial Intelligence (AI) is rapidly integrating into various aspects of our daily lives, and education is no exception. From smart devices to sophisticated software, AI is reshaping how we teach, l

SECTION 1: Tailored Learning Paths
One of the most profound impacts of AI in education is its ability to deliver highly personalized learning experiences. Traditional classrooms often struggle to cater to the diverse needs and learning

SECTION 2: Empowering Educators and Streamlining Tasks
While AI focuses on student learning, it also plays a crucial role in empowering educators. Teachers often spend considerable time on administrative tasks like grading papers, monitoring attendance, an

SECTION 3: Making Learning More Accessible and Engaging
AI is also revolutionizing how educational content is delivered, making it more accessible and engaging than ever before. For students with disabilities, AI-driven tools can offer real-time language tr

CONCLUSION:
The integration of Artificial Intelligence into education is more than a technological upgrade; it's a paradigm shift towards a more efficient, equitable, and engaging learning future. By personalizing
=====
```

Figure 2: User entering the blog topic

Output

For the input "Artificial Intelligence in Education", the application generated a blog titled:

"Learning Smarter: How AI is Reshaping Education"

The generated article contained sections discussing personalized learning, AI assistance for teachers, accessibility and engagement, followed by a conclusion.

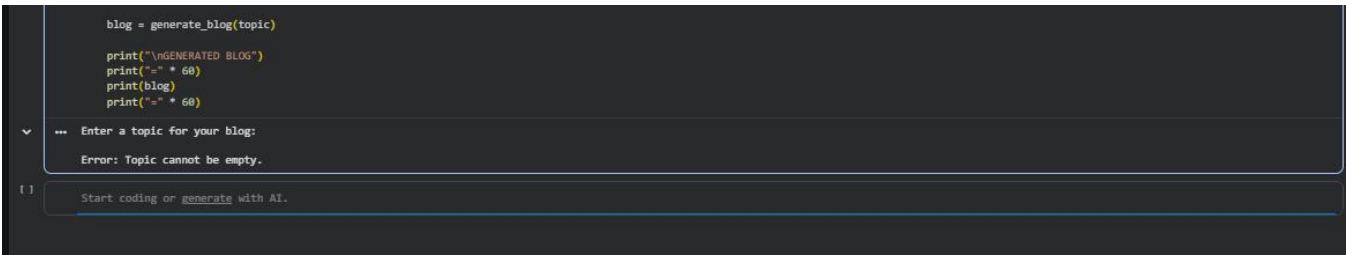
Error Handling

The application handles the following cases:

Case 1: Empty Input

If the user does not enter a topic:

If an error occurs while communicating with the Gemini API, the program catches the exception and displays an appropriate API error message instead of terminating unexpectedly.



```
blog = generate_blog(topic)
print("\nGENERATED BLOG")
print("-" * 60)
print(blog)
print("-" * 60)
```

Enter a topic for your blog:

Error: Topic cannot be empty.

Start coding or generate with AI.

Figure 4: Empty input validation

Test Cases

Test Case	Input	Expected Result	Status
TC01	Artificial Intelligence in Education	Blog generated successfully	Pass
TC02	Cybersecurity for College Students	Blog generated successfully	Pass
TC03	Empty input	Error message displayed	Pass
TC04	API unavailable/invalid API configuration	API error handled	Pass

Result

The Blog Content Generator was successfully implemented using Python and the Google Gemini API. The application accepts a topic from the user and generates a structured blog article automatically. Input validation and API error handling were also implemented successfully.

Conclusion

The project demonstrates how a Large Language Model API can be integrated into a Python application to automate content generation. The system provides a simple interface for users to enter a topic and receive an informative blog article. The implementation also demonstrates basic error handling, modular programming, and API-based generative AI.

25. College FAQ Retrieval System: Develop a semantic search system for college-related FAQs using embeddings and a vector database. Retrieve the top relevant answers for user queries.

College FAQ Retrieval System Using Embeddings and Vector Database

Aim

To develop a College FAQ Retrieval System that uses text embeddings and a vector database to semantically search college-related frequently asked questions (FAQs) and retrieve the most relevant answers for a user's query.

Objective

The main objectives of this application are:

- To create a knowledge base containing college-related FAQs.
- To convert FAQ questions into numerical vector representations called embeddings.
- To store the embeddings in a vector database.
- To accept natural-language questions from users.
- To perform semantic similarity search.
- To retrieve the top relevant FAQs for the user's query.
- To display the retrieved answers and their relevance scores.
- To handle queries for which no sufficiently relevant FAQ is available.

Technologies Used

Technology	Purpose
Python	Application development
Google Gemini API	Embedding/AI processing
FAISS	Vector database and similarity search
Google Colab	Development and execution
NumPy	Numerical vector operations

Problem Description

College students frequently need information about areas such as fees, examinations, library facilities, attendance, certificates, departments, and academic procedures. Searching through a large collection of FAQs using exact keywords may fail when the user's wording differs from the stored question.

The proposed system uses semantic search to understand the meaning of the user's query and retrieve FAQs that are semantically similar, even when the exact words are different.

Working Principle

The system works in two major stages.

Stage 1 – Creating the FAQ Vector Database

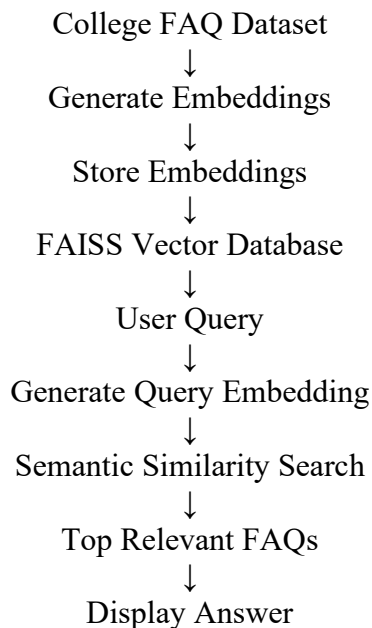
1. A collection of college-related FAQs is created.
2. Each FAQ question is converted into an embedding.

3. The generated embeddings are stored in a vector database using FAISS.
4. The corresponding answers are stored along with the FAQ information.

Stage 2 – Retrieving an Answer

1. The user enters a college-related question.
2. The user's question is converted into an embedding.
3. The system compares the query embedding with the stored FAQ embeddings.
4. Similarity scores are calculated.
5. The top relevant FAQs are retrieved.
6. The corresponding answers are displayed to the user.

System Flow



What is an Embedding?

An embedding is a numerical representation of text that captures its semantic meaning.

For example:

"What are the library timings?" and "When can students use the library?" use different words but have a similar meaning.

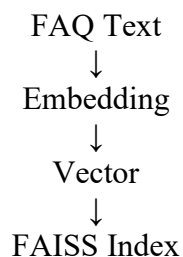
Their embeddings should therefore be close to each other in the vector space. This allows the system to retrieve relevant FAQs even when the user's wording is different.

Vector Database

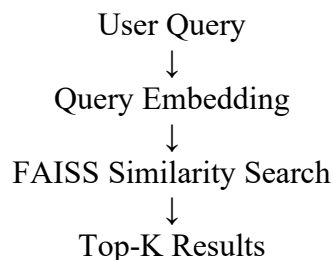
A vector database stores numerical vector representations of data and allows efficient similarity searches.

In this application, FAISS (Facebook AI Similarity Search) is used to store and search the FAQ embeddings.

The basic process is:



When a user asks a question:



FAQ Knowledge Base

A small college FAQ knowledge base is created for the system.

Example:

FAQ	Question	Answer
FAQ 1	What are the college working hours?	The college working hours are from 8:30 AM to 4:30 PM.
FAQ 2	How can I apply for a bonafide certificate?	Students can apply for a bonafide certificate through the designated student service process.
FAQ 3	What are the library timings?	The library is available during the prescribed college working hours.
FAQ 4	How can I pay my college fees?	Students can pay their fees through the available college fee payment facility.
FAQ 5	How can I contact the examination department?	Students can contact the examination department through the official college communication channels.

Note: Replace these sample answers with your actual college-approved FAQ information if your faculty expects real college data.

Semantic Search

Unlike traditional keyword search, semantic search focuses on the meaning of the query.

For example:

Stored FAQ:

How can I obtain a bonafide certificate?

User query:

I need a bonafide. What should I do?

Although the two sentences are not identical, they have similar meanings. The embedding-based search can identify the stored FAQ as a relevant result.

Retrieval Process

Suppose the user enters:

How do I get a bonafide certificate?

The system generates an embedding for the query and searches the FAISS index.

Example output:

User Query:

How do I get a bonafide certificate?

Top Relevant FAQs:

1. How can I apply for a bonafide certificate?
Similarity Score: 0.91
2. How can students request college certificates?
Similarity Score: 0.78
3. Where can students access student services?
Similarity Score: 0.64

The system then displays the answer associated with the most relevant FAQ.

Input

The system accepts a natural-language college-related question.

Example:

How do I get a bonafide certificate?

Another example:

When can I use the college library?

Output

Example:

Query:
How do I get a bonafide certificate?

Most Relevant FAQ:
How can I apply for a bonafide certificate?

Answer:
Students can apply for a bonafide certificate through the designated student service process.

The system can also display the similarity score to demonstrate the retrieval process.

Handling Irrelevant Queries

The system should also handle questions that are unrelated to the available FAQs.

Example:

Query:
What is the weather today?

If no FAQ has a sufficiently high similarity score, the system should display:

No sufficiently relevant FAQ was found.
Please try asking a college-related question.

This prevents the system from returning an unrelated FAQ.

Test Cases

Test Case	User Query	Expected Result	Status
TC01	How do I get a bonafide certificate?	Relevant bonafide FAQ retrieved	Pass
TC02	When can I use the library?	Relevant library FAQ retrieved	Pass
TC03	How can I pay my college fees?	Relevant fee FAQ retrieved	Pass
TC04	What is today's weather?	No relevant FAQ message	Pass

Result

The College FAQ Retrieval System was successfully designed using embeddings and a vector database. The system accepts natural-language college-related questions, performs semantic similarity search, retrieves the top relevant FAQs, and displays the corresponding answers.

Conclusion

The project demonstrates how embeddings and vector databases can be used to build an efficient semantic search system. Unlike simple keyword-based searching, the system can identify FAQs based on the meaning of a user's query. This makes it useful for providing quick access to college-related information.

RUBRICS FOR CALCULATION

Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
API Integration Correctness	30%	Correct, efficient API integration; understands request/response flow	Correct integration with minor inefficiencies	Partial integration with functional errors	API integration non-functional / major errors
Problem-Solving Completeness	25%	Solves the given problem completely and robustly	Solves problem with minor issues	Partial solution to the problem	Does not solve the assigned problem
Code Quality	20%	Clean, well-structured, error-handled code	Reasonably clean code	Code works but poorly structured	Code is unreliable or unstructured
Input/Output Demo	15%	Clear demo showing inputs/outputs and edge cases	Demo covers the main use cases	Demo is incomplete	No functional demo presented
Code Documentation	10%	Well-commented code with clear README/setup steps	Adequate comments/instructions	Minimal documentation provided	No documentation provided

Marks Evaluation:

Question No.	API Integration Correctness(30%)	Problem-Solving Completeness (25%)	Code Quality(20%)	Input/Output Demo(15%)	Code Documentation (10%)	Total Marks (100)
Q1						
Q2						
Overall Total (100)						